基于 TuGraph 的真实邮件网络图查询与图算法实验报告

专业综合实践 · 实验 2 · GitHub 账号：Ruinan-Li · 整理日期：2026-10-30

摘要

本实验按照课程 PPT“实验2-图查询与图算法”的要求，采用 Stanford SNAP 发布的 email-Eu-core 真实邮件通信网络，在 TuGraph 4.5.2 上完成属性图建模、数据导入、Cypher 查询、增删改查、中心性计算、图遍历、社区检测和节点嵌入。原始关系文件包含 25,571 行数据，超过课程要求的 1 万行。清洗仅删除 642 条自环，保留全部 1,005 个节点和 24,929 条有向通信关系。

PageRank、BFS、标签传播及 Node2Vec 偏置随机游走都通过 TuGraph 内的 C++ 存储过程运行。客户端使用真实游走序列训练 64 维 Skip-gram 向量，并用真实部门标签完成传导式节点分类。五组随机种子下，Node2Vec 的平均 Accuracy 为 0.7478，Macro-F1 为 0.6285，Accuracy 比五维结构特征基线高 63.64 个百分点。标签传播在本次设置下将最大连通分量合并为一个社区，该负面结果被保留并分析。统计与图表来自已完成的真实运行；公开附件范围与原始证据留存限制详见第12节。

1. 实验任务与完成范围

PPT 第 2 页明确要求：选择达到规模要求的数据集，说明来源、功能和结构，基于 TuGraph 完成图分析，至少使用两种图算法且其中一种必须是节点嵌入，最后提交网页链接或网页导出的 PDF。PPT 第 3 页起依次介绍建模、Cypher、路径、遍历、中心性、社区、相似性与嵌入。本实验围绕同一份真实网络串联这些环节。

课程内容	本实验的实现	证据文件
公开数据不少于1万行	25,571行原始关系，保留原始压缩包	data/manifest.json
属性图建模与导入	Person 节点、EMAIL 有向边、主键索引	data/schema.json
基础、聚合及多跳查询	节点/边计数、度排序、部门流量、两跳路径	results/cypher_results.json
增删改查	主图属性 SET/REMOVE；隔离子图真实记录 CREATE/DELETE	results/cypher_results.json
图中心性	TuGraph 内 PageRank 并回写节点属性	results/pagerank.json
遍历和路径	TuGraph 内 BFS，独立核对最短距离	results/bfs.json
社区检测	五组种子的异步标签传播	results/lpa_metrics.csv
节点嵌入	TuGraph 采样 + Skip-gram 训练 + 部门分类	results/classification_metrics.csv
报告与核心代码	中文报告、图表、存储过程、运行脚本和日志	README.md、scripts/、procedures/

发布说明：表中的路径为原运行时文件名，并不表示所有文件均随本分支发布。完整报告与7张图表已恢复；部分未上传脚本、逐节点结果和日志已无法恢复。请结合第12节阅读。

2. 数据来源、含义及真实性

2.1 原始数据

数据来源为 SNAP email-Eu-core 官方数据页。网络来自某欧洲研究机构内部人员之间的匿名邮件通信。节点代表匿名成员，原始有向边 u→v 表示成员 u 至少向成员 v 发送过邮件。部门标签文件给出每个成员所属的真实部门编号，共 42 类。原始文件不含姓名、邮箱正文或身份信息，本实验不对匿名编号进行现实身份推断。

该数据属于静态二值通信网络。单条边不能解释成“仅发送一封邮件”，也没有权重、时间戳、邮件内容或持续时间。实验中的部门流量表示不同成员对之间存在的通信关系数，不是邮件封数。原始有向图中的 u→v 与 v→u 是不同记录，不能在导入阶段合并。

数据包含两个原始文件：关系文件每行两列 SRC、DST，标签文件每行两列 NODEID、DEPARTMENT。下载文件按字节保留，SHA-256 可用于检查数据版本和传输完整性。

原始文件	数据行数	字节数	SHA-256
email-Eu-core.txt.gz	25571	79754	4b47acdb80197b085fe63c819c357ae488131ee904ed93d1b219a68b0f9e245f
email-Eu-core-department-labels.txt.gz	1005	2663	e5abe5b4581a480032a63adcf2576c161785f45692642c6ebb0b1276f0f33669

2.2 清洗流程及导入检查

先按整数读取两列数据，确认全部边端点都存在部门标签，再检测完全重复的有向边、自环及节点覆盖范围。实际重复记录为 0 条，自环为 642 条。自环保留在原始文件中，但从分析图中删除，以免将“给自己发信”计为社会连接。没有按算法表现筛选节点或边，也没有人为增加连接。

清洗后主图包含 1,005 个节点、24,929 条有向边。保留 19 个在去除自环后没有其他邻居的节点，避免把“无外部联系”误判为数据缺失。Node2Vec 对这些节点没有可用上下文，因此训练和分类时单独处理，详见第 8 节。

导入完成后，存储过程从 TuGraph 读取每一个节点和每一条边，与预处理 CSV 做集合及属性逐项比较。results/data_verification.json 记录两项检查均为 true。检查范围不仅是节点数和边数，还包括端点对集合、节点编号以及真实部门标签。

2.3 两种图视图

1. 有向图：保留原始邮件方向，用于 Cypher 流向查询、入度/出度、PageRank 及部门关系统计。

2. 无向投影：只要任意方向存在通信，就保留一条无向边。合并后有 16,064 条无向边，用于 BFS、标签传播和 Node2Vec。该投影表达“曾有通信联系”，不再保留发送和接收方向。

两种视图共享相同的节点集合。无向投影在存储过程读出邻接关系后构建，不修改数据库中的原始方向。比较算法时必须区分图视图，不能把有向边数与无向边数当成数据丢失。

3. 环境与整体执行流程

实际执行日期以 results/environment.json 的 UTC 时间为准：2026-10-30T06:46:45.060080+00:00。本次使用 CPU 运行，未使用 GPU。数据库版本为 4.5.2，客户端 Python 为 3.12.14，执行平台为 Linux-6.18.57-x86_64-with-glibc2.41。TuGraph 官方二进制显示其编译时 Python 为 "3.6.9"，这一版本与客户端训练环境分离。

服务只监听 127.0.0.1，HTTP 端口 7070、RPC 9090、Bolt 7687。本实验实际通过 REST 提交 Cypher 和存储过程请求。认证开启，口令从环境变量读取，不将 JWT 或用户口令写入结果。使用独立子图 email_eu_lab2_20261030，CRUD 删除操作位于另一个实验子图中，不清空默认数据库。

执行流程为：下载并校验原始数据；清洗并生成 CSV；创建 Schema 并批量导入；执行 Cypher 查询；注册只读 C++ 存储过程；运行图算法和 Node2Vec 采样；训练嵌入并分类；独立核验算法输出；回写中心性和社区属性；绘制图表并生成报告。服务启动、数据库分析与正常关闭都由脚本管理。

3.1 为什么采用 C++ 存储过程

提供的 ceshi_node2vec.py 使用 TuGraph 只读事务和出边迭代器实现偏置随机游走。其输出是节点访问序列，还不是嵌入向量。为完成整图多种子实验，本项目把相同的数据库读取与偏置采样机制扩展为 C++ 存储过程，并增加完整向量训练与评估。这也避免把官方安装包的 Python 3.6 扩展直接装入 Python 3.12.14 客户端造成 ABI 不兼容。

存储过程的图输入来自 GraphDB.CreateReadTxn()、顶点迭代器和边迭代器。PageRank、BFS、LPA 与 Node2Vec 采样均在 TuGraph 服务进程中执行。NetworkX 仅用于独立结果校验和辅助统计，Gensim 用于客户端 Skip-gram 训练，不能把这两部分的耗时表述成数据库内嵌训练性能。

4. 属性图建模与导入

元素	字段/约束	类型	意义
Person	person_id，主键	INT64	SNAP 原始匿名成员编号
Person	department	INT32	真实部门编号，仅作属性查询和评估标签
Person	note，可空	STRING	增改删属性的操作记录
Person	pagerank，可空	DOUBLE	算法运行后回写的中心性
Person	community，可空	INT64	seed=42 的标签传播结果
EMAIL	Person→Person	有向关系	一对成员之间观察到的邮件通信
EMAIL	无权重字段	-	源数据没有邮件次数，禁止补造权重

person_id 是业务主键，TuGraph 内部 VID 只是数据库存储编号。实现首先构建 VID 到连续数组位置的映射，所有结果再还原为 person_id，没有假设内部 VID 必然等于原始节点编号。

采用 db.importor.schemaImportor 创建 Schema，再用 db.importor.dataImportor 导入节点 CSV 和边 CSV。必须先导入节点，再导入边。TuGraph 4.5.2 的这两个接口要求传入 Base64 编码的描述和数据内容，导入配置使用 continue_on_error=false，避免静默跳过坏行。节点 CSV 带表头 person_id,department，边 CSV 带表头 src,dst，均设置 header=1。

```
description = json.dumps({"files": [file_spec]})
content = csv_path.read_text()
query = (
  "CALL db.importor.dataImportor("
  f"'{enc(description)}','{enc(content)}',false,1,',')"
)
client.cypher(query)
```

5. Cypher 图查询与增删改查

下面所有查询均已实际提交 TuGraph，原始响应包含列名、结果行、数据库 elapsed 和客户端耗时，保存于 results/cypher_results.json。报告选取代表性结果展示。

5.1 节点、边与分组计数

```
MATCH (n:Person) RETURN count(n) AS persons;
MATCH (:Person)-[r:EMAIL]->(:Person) RETURN count(r) AS emails;
MATCH (n:Person)
RETURN n.department AS department, count(n) AS persons
ORDER BY persons DESC;
```

前两条分别返回 1,005 和 24,929。第三条按 department 隐式分组，返回 42 行。RETURN 中的非聚合字段成为分组键，Cypher 不需要显式 GROUP BY。

5.2 出度、入度与部门通信

```
MATCH (p:Person)-[:EMAIL]->(q:Person)
RETURN p.person_id AS person_id, p.department AS department,
    count(q) AS degree
ORDER BY degree DESC, person_id LIMIT 10;

MATCH (p:Person)-[:EMAIL]->(q:Person)
WHERE p.department <> q.department
RETURN count(*) AS cross_department_edges;
```

出度表示不同收件人的数量，入度表示不同发件人的数量。排序中加入 person_id，使度数相同节点的展示次序明确。跨部门查询返回 16,284 条关系，而部门内部关系为 8,645 条。

节点	部门	出度
160	36	333
82	36	226
121	36	221
107	36	203
86	36	201
62	36	189
13	26	171
249	14	159
183	4	158
434	34	156

5.3 两跳连接

```
MATCH (p:Person {person_id:0})-[:EMAIL]->(q:Person)-[:EMAIL]->(r:Person)
WHERE p.person_id <> r.person_id
RETURN r.person_id AS person_id, count(DISTINCT q) AS intermediaries
ORDER BY intermediaries DESC, person_id LIMIT 10;
```

这条查询计算节点 0 经两个有向步骤可到达的成员，以及不同中间人的数量。它并未排除一跳邻居，所以结果只能称为“两跳可达与中介数量”，不能直接称为新增联系人推荐。下表保留实际返回的前十个节点。

目标节点	不同中介数
74	20
215	19
221	18
17	17
223	17
86	16
177	15
218	15
1	14
222	14

5.4 创建、修改和删除

主图上先对真实节点 0 设置 note='Lab 2 verification'，查询确认后使用 REMOVE p.note 清除该可空属性，保留原有人员和通信关系。随后在独立 CRUD 子图中，从源数据取出第一条真实边及其两个真实端点，执行 CREATE、DELETE 关系、DELETE 节点，并检查关系和节点均回到 0。未使用虚构人员或虚构通信边。

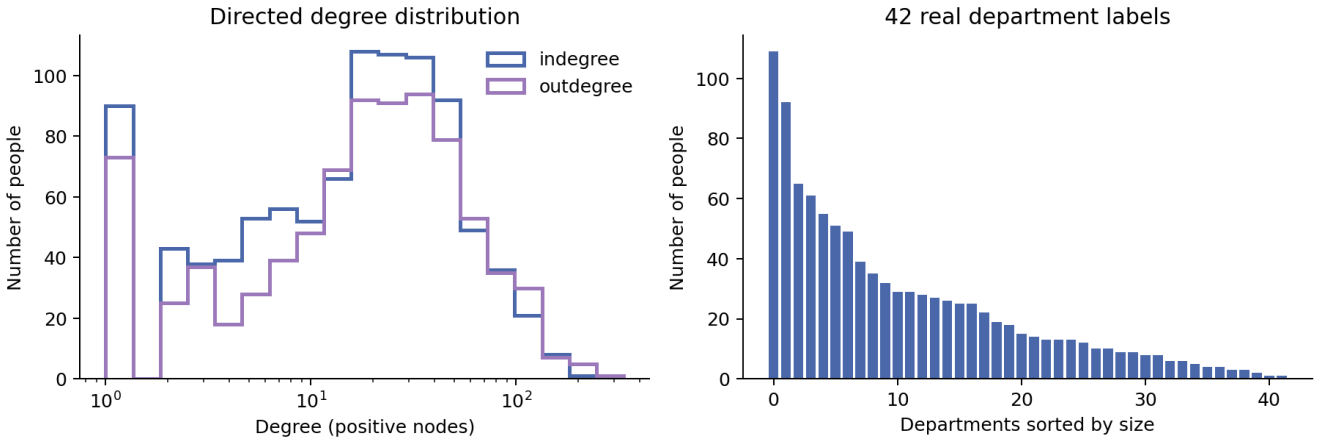
```
MATCH (p:Person {person_id:0})
SET p.note='Lab 2 verification' RETURN p.person_id,p.note;
MATCH (p:Person {person_id:0})
REMOVE p.note RETURN p.person_id,p.note;
MATCH (a)-[r:EMAIL]->(b) DELETE r;
MATCH (n) DELETE n;
```

最后两句只在独立 CRUD 子图执行。实际执行语句及其所属 graph 字段可在查询日志中核查。DELETE 删除实体，REMOVE 删除可空属性，两者用途不同。

6. 网络结构与中心性分析

6.1 基础结构

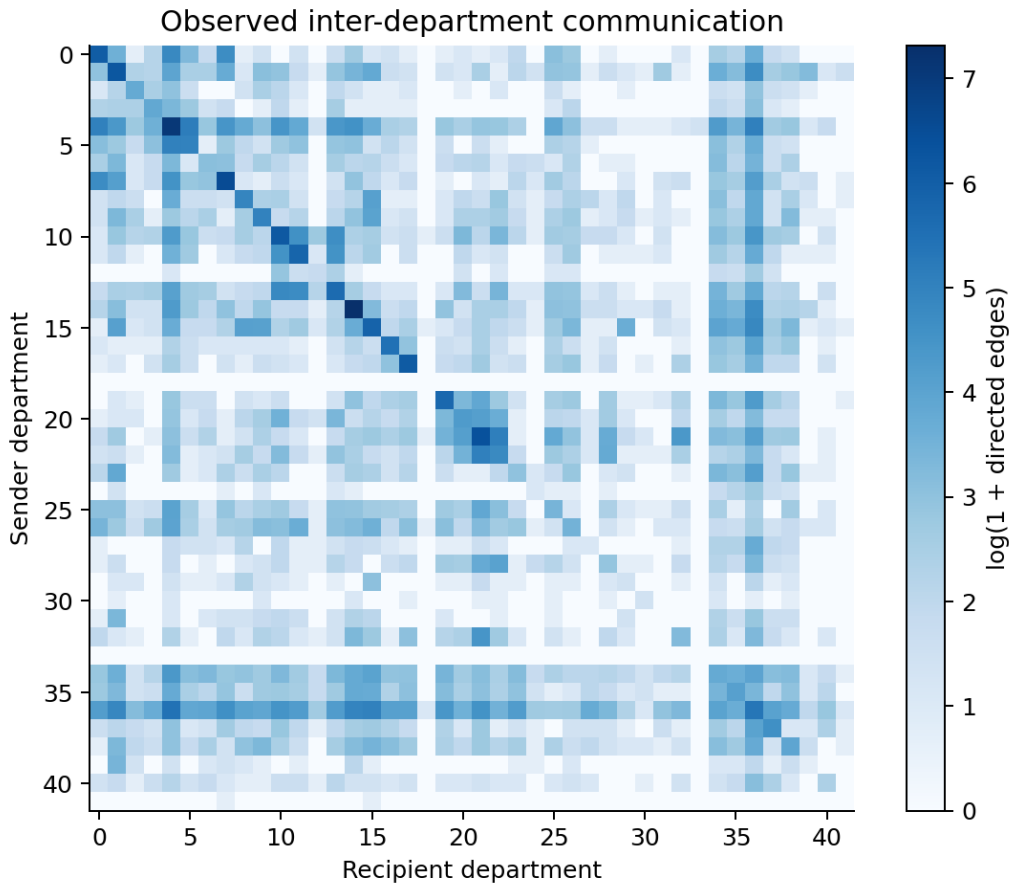
指标	实际结果
节点数	1005
有向边数	24929
无向投影边数	16064
弱连通分量	20
最大弱连通分量	986
强连通分量	203
最大强连通分量	803
去自环后孤立节点	19
有向密度	0.024706
有向互惠率	0.711220
无向平均聚类系数	0.399355



节点度分布和真实部门规模

图 1 展示不同人员的连接数量以及部门规模差异。度分布图只绘制正度数，19 个无向孤立节点仍保留在统计和数据库中。部门规模不均衡，因此分类报告同时给出 Accuracy 和 Macro-F1，避免只看大部门的预测效果。

互惠率 0.7112 表示有向边中有反向对应边的比例，不能解释成邮件回复率。无向平均聚类系数描述邻居之间闭合三角形的程度。本图的最大弱连通分量包含 986 人，其余为 19 个孤立点。



部门之间观察到的通信关系

图 2 的行是发送方部门、列是接收方部门，颜色为 log(1+关系数)。对数仅用于可视化，原始 42×42 计数保存在 results/department_flow_matrix.csv。未进行按部门规模归一化，故大部门的高流量可能同时受人数影响。跨部门关系比例为 65.32%，这一结构可能促使简单标签传播产生大社区，但该观察不能单独证明算法退化的全部原因。

6.2 PageRank 原理与实现

PageRank 在有向图上衡量收件节点的重要性：从高重要性发件人获得连接，会提升接收节点的得分。每轮把节点的分数沿其出边平均传播，并加入均匀随机跳转。阻尼系数 alpha=0.85；初值为 1/N；没有出边的节点把得分均匀分配给全部节点。迭代使用以下更新式：

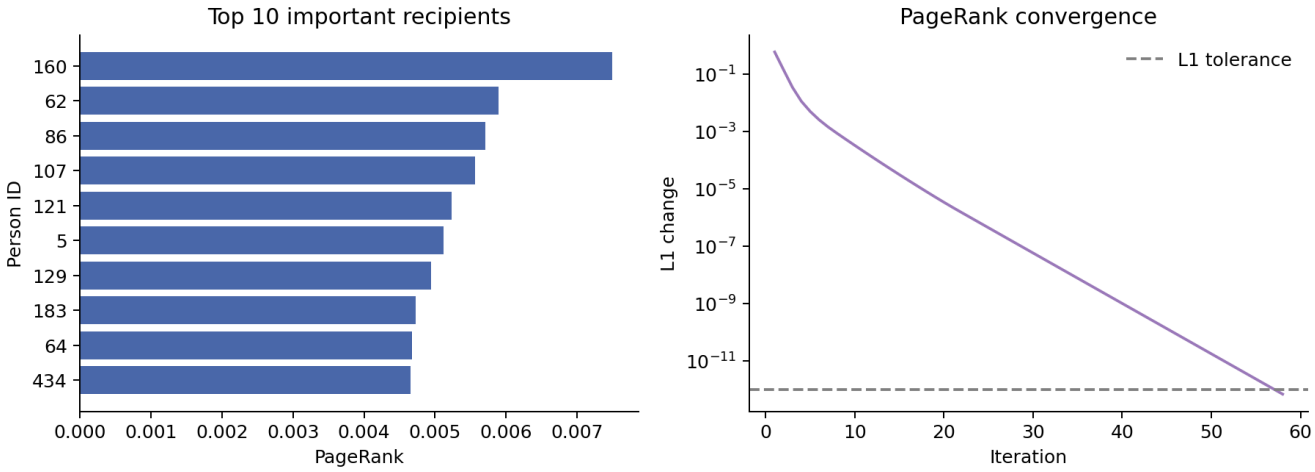
PR_new(v) = (1-alpha)/N + alpha*dangling_mass/N + alpha*sum(PR(u)/outdegree(u))

求和范围为指向 v 的节点 u。停止条件是相邻两轮分数向量的 L1 差小于 1e-12，最多迭代 500 次。算法只依赖边结构，不使用部门标签。主要计算复杂度为 O(K(N+M))，其中 K 是迭代次数。

实际在 58 轮达到收敛，最后 L1 变化为 6.984e-13，得分总和为 1.000000000000。独立 NetworkX 实现与数据库结果的最大绝对误差为 2.203e-13。这一检查确认了方向、悬挂节点处理和归一化规则的一致性。

排名	节点	部门	PageRank	入度	出度
1	160	36	0.00749615	211	333
2	62	36	0.00589415	178	189
3	86	36	0.00570852	153	201
4	107	36	0.00556441	168	203

排名	节点	部门	PageRank	入度	出度
5	121	36	0.00523139	156	221
6	5	25	0.00511705	123	155
7	129	4	0.00494818	138	135
8	183	4	0.00472662	142	158
9	64	25	0.00467315	135	112
10	434	34	0.00465310	150	156



PageRank前十名与收敛曲线

最高分节点为 160。PageRank 与入度的 Spearman 相关系数为 0.982871，说明本图上二者排序高度相关，但 PageRank 还利用邻居的重要性。由于成员身份匿名，本报告只讨论其网络结构位置，不把高分解释为行政级别或工作绩效。计算结果已回写 Person.pagerank，并通过 Cypher 重新读取前十名确认。

7. BFS 路径与标签传播社区检测

7.1 BFS

以 PageRank 最高的节点 160 为起点，在无向投影上使用队列进行广度优先遍历。第一次访问节点时记录距离和父节点，因此得到的是无权图最少边数路径。时间复杂度为 O(N+M)，访问数组防止重复入队。

实际可达 986 个节点。距离分布为：0 跳 1 人, 1 跳 345 人, 2 跳 585 人, 3 跳 51 人, 4 跳 4 人。一条最短路径示例是 160 → 27 → 370 → 414 → 449，长度为 4 条边。此处是无向通信距离，不等于邮件沿发送方向转发的路径。最大距离仅针对该起点，不作为全图直径。

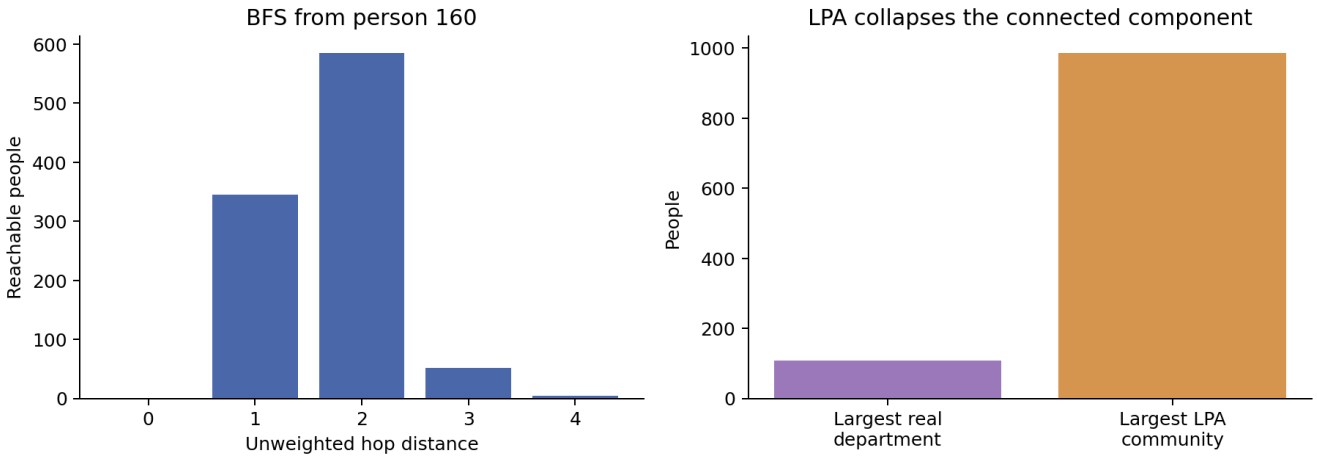
BFS 距离数组已与 NetworkX 单源最短路逐节点核对一致，父节点与访问顺序保存于 results/bfs.json。

7.2 标签传播原理

每个节点初始拥有不同标签。每轮随机打乱节点访问顺序，统计邻居标签并选择出现次数最多的标签，采用异步更新。若当前标签已处于并列最高频，就保持当前标签；否则在最高频候选中随机选择。全部节点标签不再改变时停止，最多 100 轮。孤立节点保留各自初始标签。部门真值不参与标签传播。

随机种子为 11、23、42、71、101。除模块度 Q 外，同时使用 NMI 和 ARI 比较社区与真实部门的一致性。ARI 可以小于 0，负值并不意味着计算错误，而是说明相对于其随机校正基准，划分未表现出正向一致性。

种子	社区数	最大社区	迭代轮数	模块度	NMI	ARI
11	20	986	6	0.000000	0.032919	-0.000732
23	20	986	6	0.000000	0.032919	-0.000732
42	20	986	6	0.000000	0.032919	-0.000732
71	20	986	4	0.000000	0.032919	-0.000732
101	20	986	7	0.000000	0.032919	-0.000732



BFS距离和标签传播合并现象

五次运行均得到 20 个社区，其中一个包含全部 986 个非孤立节点，另外 19 个分别对应孤立节点。这个“20”不代表找到了 20 个有效业务社区。最大连通分量内部没有再被划分，模块度为 0。NMI 仅约 0.033，主要不能视作部门恢复成功的证据。真实部门标签在同一无向图上形成的模块度为 0.288013，说明图中仍存在可用于部门分析的结构，而本次 LPA 设置没有将其有效区分。

该负面结果被完整保留。它提示：收敛只是算法停止条件，并不保证社区具有解释力。后续可尝试 Louvain、Leiden、带约束或带分辨率参数的算法，再按相同图视图比较。本报告没有把这些尚未执行的方法列入结果，也未修改标签传播输出以使其更接近部门标签。

8. Node2Vec 节点嵌入

8.1 从提供的游走代码扩展到完整实验

原始 Python 示例针对一个起点生成若干游走，读取的是出邻居，并返回字符串形式的列表。本项目扩展为：遍历全部非孤立节点；固定随机种子；每个节点重复 10 次；显式采用去重无向邻接；检查 p、q 和长度参数；使用 JSON 返回结构化结果；保存完整游走序列；完成 Skip-gram 训练及下游评估。

首步在起点邻居中均匀采样。从第二步开始，设上一节点为 t、当前节点为 v、候选邻居为 x，未归一化权重为：返回 t 时 1/p；x 与 t 相邻时 1；其他情况 1/q。对 v 的全部候选邻居归一化后采样。p 大时抑制立即返回；q 小于 1 时增加向更远位置探索的相对权重；q 大于 1 时更偏向局部。原代码中对 BFS/DFS 的注释应以这个实际转移规则为准。

```
weight = (x == previous) ? 1.0 / p
    : (adjacent(previous, x) ? 1.0 : 1.0 / q);
next = neighbors[discrete_distribution(weights)(rng)];
```

其中 adjacency 来自 TuGraph 实际边。随机数只用于真实图上的算法采样，不用于生成网络记录或替代测量结果。

8.2 参数设置

参数	取值/规则
图视图	去自环、去重无向投影
起点范围	986 个非孤立节点
每节点游走次数	10
每条游走长度	40 个节点
主实验 p、q	p=1，q=0.5
敏感性对照	seed=42，q=1 和 q=2
随机种子	11、23、42、71、101
向量维数	64
训练目标	Skip-gram + negative sampling
上下文窗口	5
负采样数	5
训练轮数	10
高频词下采样	sample=0.001
min_count	1
训练工作线程	1
稳定初始化	固定种子、稳定 SHA-256 token 哈希
评估模型	StandardScaler + LogisticRegression(C=1, max_iter=2000)

每个配置产生 9,860 条游走、394,400 个节点 token，共训练 7 个嵌入配置。验证脚本检查每条游走长度、每个非孤立节点起点次数，以及每一步是否确实对应输入图中的边。所有检查通过。19 个孤立节点不生成无意义的重复 token，也不填充随机向量。

Skip-gram 把中心节点和其游走窗口中的上下文作为训练样本，学习能够预测上下文的节点向量。负采样是模型训练目标中的抽样机制，不是虚构观察边。部门标签不进入游走转移、词表构建或嵌入训练。

```
model = Word2Vec(
  sentences=walks_as_tokens, vector_size=64, window=5,
  min_count=1, sg=1, hs=0, negative=5, sample=1e-3,
  workers=1, epochs=10, seed=seed, hashfxn=stable_hash
)
```

全部 986 个非孤立节点均获得有限的 64 维向量，按 person_id 与矩阵行一一对应保存为 NPZ。训练中的单线程和固定哈希减少随机扰动，但跨编译器、BLAS及库版本不能承诺位级完全相同。本次曾保存完整游走与向量；现有公开交付范围及原文件留存限制见第12节。

8.3 传导式分类协议与数据泄漏控制

分类任务是根据图结构预测真实部门。完整图的拓扑对训练可见，只有训练节点的部门标签用于监督分类，因此属于传导式节点分类。它不代表预测从未出现的新节点，也不代表预测未来邮件。

先排除孤立节点，再排除非孤立成员少于 5 人的部门，使分层划分有足够样本。实际纳入 968 个节点、35 个部门，排除 37 个节点。排除名单、原因及每次训练/测试节点编号全部记录于 results/evaluation_protocol.json，报告分数仅适用于这部分节点。

每个种子使用 70%/30% 分层划分，实际为 677 个训练节点、291 个测试节点。对同一个种子，所有基线使用完全相同划分。标准化器只拟合训练节点特征；测试标签不用于嵌入训练、分类器拟合、特征标准化或参数选择。C 固定为 1，没有按测试集选超参数。

对照方法包括：Majority，只预测训练集最多的部门；Structural，使用 log(1+入度)、log(1+出度)、log(1+无向度)、PageRank、局部聚类系数五维结构特征；Node2Vec，使用 64 维向量。后两者使用相同分类器和标准化流程。

Accuracy 为预测正确的节点比例。Macro-F1 先计算每个部门的 F1，再对部门等权平均，适合检查小部门。Weighted-F1 按测试集各部门人数加权。单标签多分类中的 Micro-F1 与 Accuracy 相等，因此未重复展示。

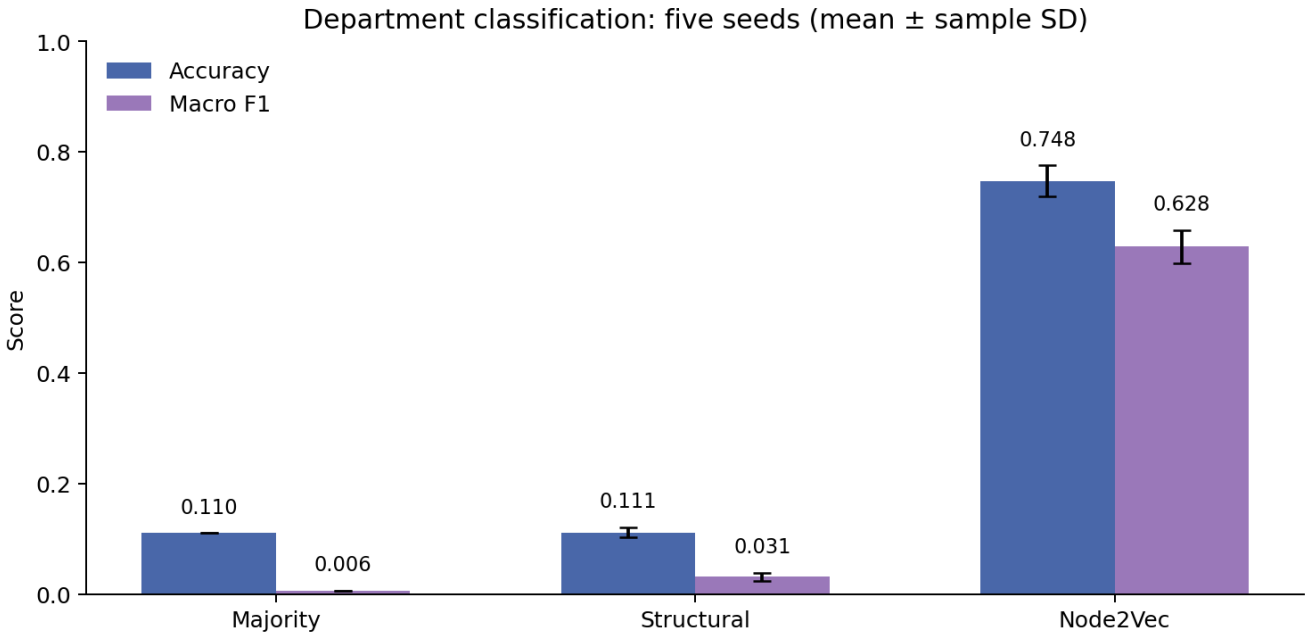
9. 节点分类、参数对照与相似节点

9.1 五组种子的完整分类结果

种子	方法	Accuracy	Macro-F1	Weighted-F1
11	Node2Vec	0.7938	0.6781	0.7872
11	Structural	0.1100	0.0276	0.0547
11	Majority	0.1100	0.0057	0.0218
23	Node2Vec	0.7491	0.6320	0.7402
23	Structural	0.1203	0.0390	0.0713
23	Majority	0.1100	0.0057	0.0218
42	Node2Vec	0.7388	0.6176	0.7291
42	Structural	0.1134	0.0280	0.0576
42	Majority	0.1100	0.0057	0.0218
71	Node2Vec	0.7216	0.6141	0.7224
71	Structural	0.1168	0.0380	0.0677
71	Majority	0.1100	0.0057	0.0218
101	Node2Vec	0.7354	0.6007	0.7330
101	Structural	0.0962	0.0216	0.0500
101	Majority	0.1100	0.0057	0.0218

各方法的均值与样本标准差（ddof=1，n=5）如下。标准差同时包含游走、嵌入训练及标签划分改变的影响，不是固定测试集上的纯模型随机性。

方法	Accuracy	Macro-F1	Weighted-F1
Majority	0.1100 ± 0.0000	0.0057 ± 0.0000	0.0218 ± 0.0000
Structural	0.1113 ± 0.0093	0.0308 ± 0.0074	0.0603 ± 0.0089
Node2Vec	0.7478 ± 0.0276	0.6285 ± 0.0299	0.7424 ± 0.0259



五组种子的节点分类比较

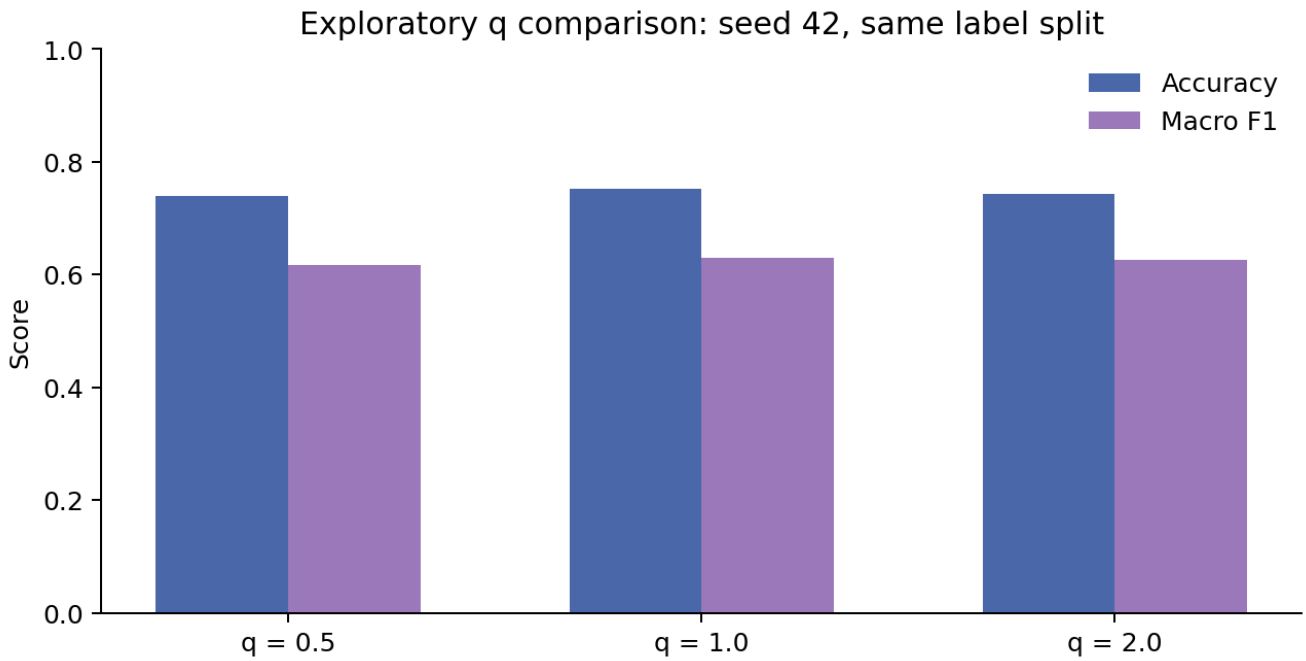
Node2Vec 相对 Structural 的平均 Accuracy 提升为 63.64 个百分点，表明游走上下文提供了超出度数和中心性的部门相关结构信息。Macro-F1 低于 Accuracy，提示稀疏部门或边界节点仍较难分类。不能据此推断匿名成员的职责、岗位或组织绩效，也不能把当前排除小类别后的结果外推为全部 42 个部门的准确率。

本实验没有做显著性检验，五组结果用于观察稳定性。五次随机划分共享数据且互有重叠，不能当成五个互不相关的新数据集。

9.2 q 参数敏感性

固定 seed=42、p=1、训练设置和标签划分，仅改变 q。此项为单种子探索性比较，不参与主实验参数选择，也不足以确定通用最优 q。

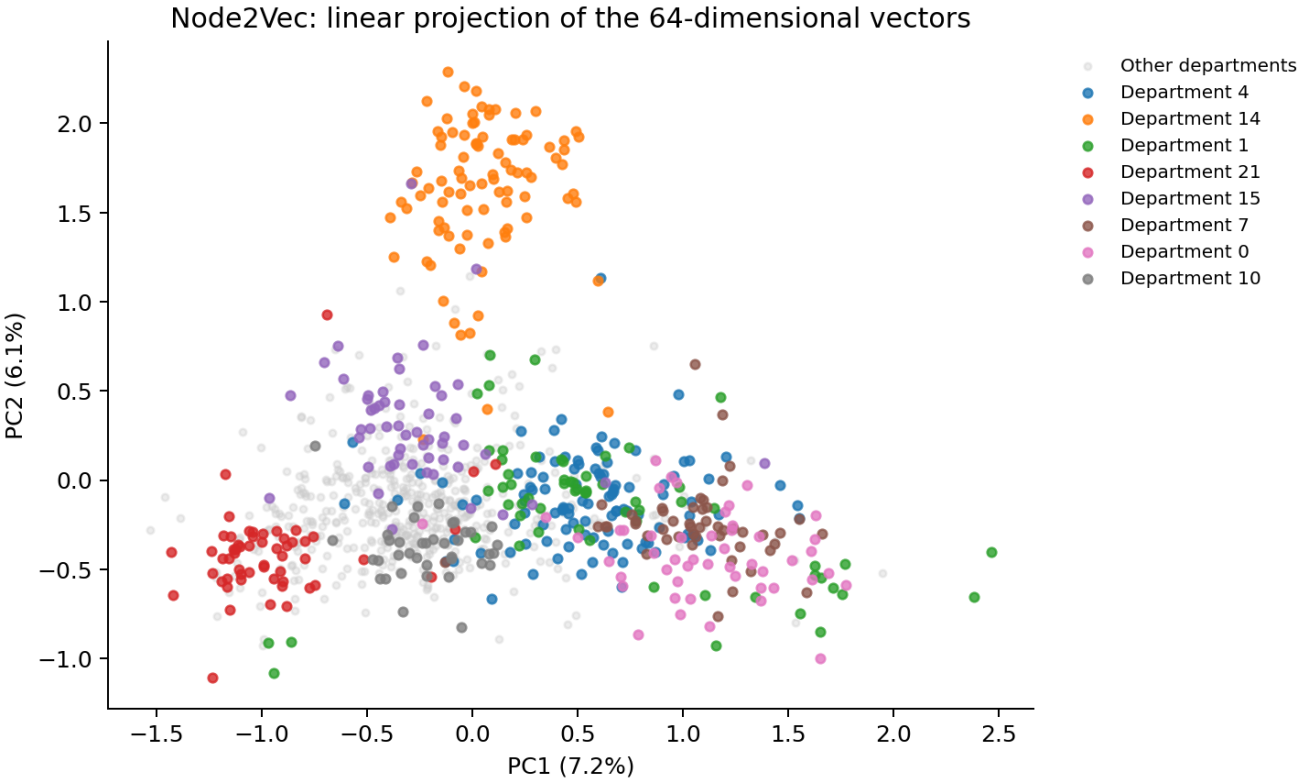
q	Accuracy	Macro-F1	Weighted-F1
0.5	0.7388	0.6176	0.7291
1.0	0.7526	0.6290	0.7401
2.0	0.7423	0.6265	0.7292



固定种子的q参数比较

q 改变对真实邻接结构的采样偏好。性能差异反映了局部连通、较远结构信息和训练随机性的共同影响。更全面的调参需要独立验证集，不能使用这里的测试结果再选择参数后重报同一测试分数。

9.3 嵌入可视化与相似节点



64维嵌入的二维PCA投影

图中是 seed=42、q=0.5 的 64 维向量经 PCA 降到二维的结果。突出展示人数最多的八个部门，其余以灰色表示。部门标签仅用于着色，PCA 不使用标签。二维投影损失高维信息，可见的重叠或分离不能替代分类评估。

以下展示 PageRank 最高节点的五个余弦最近邻。余弦相似度来自实际训练向量；“已有边”按无向投影判断。相似节点可能已经直接相连，因此这里只是相似性分析，没有将其包装为经过测试的链接预测。

锚点	锚点部门	相似节点	节点部门	余弦	同部门	已有边
160	36	166	36	0.5478	True	True
160	36	62	36	0.5463	True	False
160	36	107	36	0.5411	True	True
160	36	533	35	0.5329	False	True
160	36	301	26	0.5212	False	False

10. 实际运行时间与校验

10.1 耗时口径

存储过程内部记录读取和构建邻接的时间，以及核心算法和结果对象构造时间。客户端耗时另含 REST、序列化和解析。下面是本次运行的单次测量，不能作为不同数据库之间的性能比较。Node2Vec 客户端耗时不包括后续 Gensim 训练，Gensim 训练单独列出。

任务	种子/q	读图秒	算法秒	客户端秒
pagerank	-/-	0.005561	0.002579	0.015849
bfs	-/-	0.004281	0.000126	0.007814
lpa	11/-	0.005160	0.004126	0.012830
node2vec	11/0.5	0.004613	1.037817	1.131058
lpa	23/-	0.004459	0.003321	0.011752
node2vec	23/0.5	0.006833	1.299578	1.411911
lpa	42/-	0.004760	0.003068	0.012189
node2vec	42/0.5	0.004908	1.226558	1.332183
lpa	71/-	0.005385	0.002319	0.012704
node2vec	71/0.5	0.004495	1.134434	1.218132
lpa	101/-	0.004502	0.004547	0.014347
node2vec	101/0.5	0.013024	1.259141	1.361265
node2vec	42/1.0	0.006355	1.502097	1.597245
node2vec	42/2.0	0.007301	1.475745	1.565154

种子	q	向量训练秒	有效向量数
11	0.5	26.473	986

种子	q	向量训练秒	有效向量数
23	0.5	19.889	986
42	0.5	13.103	986
71	0.5	13.559	986
101	0.5	13.823	986
42	1.0	13.049	986
42	2.0	12.564	986

10.2 校验结果

数据库节点属性及完整边集合与预处理源文件一致。PageRank 概率和接近 1，与独立实现的最大误差低于 1e-9。BFS 的每个节点距离与独立最短路实现相同。所有 LPA 运行均收敛，并逐节点确认当前社区标签位于邻居最高频标签集合中。七组 Node2Vec 游走的全部步长均对应真实边，起点覆盖完整，所有嵌入元素有限。分类拆分、逐节点预测与指标原始CSV在运行时曾保存；当前留存限制见第12节。

这些检查分别约束数据导入、算法数值、路径正确性、社区停止条件及采样合法性。没有仅凭图表外观或“接口无报错”认定实验成功。

11. 实现中的关键问题与解决

1. 安装包依赖旧版 libgfortran.so.4。本次从 Ubuntu 官方仓库取得兼容包，解压到独立 runtime 目录，并通过 LD_LIBRARY_PATH 指定，不替换系统库。

2. 官方开发头文件引用 Boost Geometry WKB 扩展。构建时补齐上游扩展，并记录确切提交 SHA，避免使用不受版本控制的临时头文件。

3. 导入接口要求 Base64。对 Schema 描述、导入描述和 CSV 数据分别编码，禁止把普通 JSON 直接当成编码输入。

4. 存储过程通过 db.plugin.loadPlugin 注册 SO，并声明只读。完整参数保存在过程调用记录中，算法读取事务完成后及时释放。

5. TuGraph 某些 UNWIND 与属性模式组合存在前端兼容问题。本实现采用明确的主键 MATCH 回写，每次写入一个已存在节点，随后通过 Cypher 读回核对。

6. 原始样例只返回游走，不能作为节点嵌入实验终点。本项目补充了向量训练、真实标签评估、基线、五种子重复实验和参数对照。

7. 小类别与孤立节点会影响分层评价。本实现提前固定排除规则，公布名单，并明确分数的适用范围。

12. 交付范围与复现限制

本报告保留已完成实验的实际测量、算法设置、图表和结论。由于会话中断后的工作区清理，原始完整附件未能全部恢复。下一页明确区分本次公开文件、未公开数据与不可恢复内容。

12.1 公开文件与证据状态

实验计算已在上一阶段真实执行。本次恢复得到原17页报告和7张PNG图表，并逐个核对其Git对象哈希；本次发布仅修订附件可用性说明，不改动实验数值和结论。

本分支包含

README.md：详细网页版报告。
report/TuGraph_Lab2_Report.pdf：本打印版报告。
figures/：7张原始实验PNG图表。
results/*_reported.*：由原PDF表格转录的汇总值，保留显示精度。
code/：报告中实际展示的Cypher及训练代码摘录。
PUBLICATION_NOTE.md：恢复过程、范围、局限及来源。
SHA256SUMS：本分支实际交付文件的SHA-256校验清单。

未公开与未恢复的内容

额外嵌入向量的公开上传被自动审批拒绝：用户授权范围明确为报告，未明确涵盖额外派生数据。本分支不包含完整向量、游走语料或逐节点预测数据。

未上传的完整C++存储过程源码、端到端脚本、完整运行日志和部分逐节点输出，在工作区清理后未能恢复。因此本分支不是完整的一键复现包。正文中旧文件路径属于原实验记录，不应作为可点击或已交付附件使用。

复现与核验的边界

公开数据可从SNAP重新下载，原文件校验值见第2节。复现实验应按第4至10节的图视图、参数、拆分与评估协议，在TuGraph 4.5.2上重新实现、执行并保存日志。代码摘录不能替代已丢失的完整程序。

汇总CSV不伪装成原始数据库输出；四位小数不能还原全部底层精度。本次没有补造日志、游走、向量或预测结果，也没有把未执行的新程序描述为原实验代码。

12.2 课程提交事项

本分支网页和PDF可作为实验报告。PPT要求的学习通选题登记、班内数据集去重尚未核查；本报告不声称已经完成这两项课程流程。

13. 结论与局限

本实验完整打通了真实数据获取、图模型构建、TuGraph 导入、查询、数据库内图算法、节点表示训练、结果核对和报告生成。邮件网络的有向结构可以识别重要接收节点，无向投影支持通信距离和局部上下文分析。Node2Vec 的多步结构表示在本次固定分类协议下优于五维结构基线，而简单标签传播出现明显社区合并，说明算法选择需要用真实标签或其他外部指标进行检验。

局限包括：数据为单个机构的静态匿名网络；没有通信次数与时间；部门标签不必等价于真实协作社区；分类排除了孤立节点与极小类别；嵌入使用全图拓扑，属于传导式任务；q 对照只有一个种子；没有独立验证集调参、显著性检验、未来链接预测或数据库性能对照。上述范围均与本次实际执行一致。进一步工作应先补充明确的评估协议，再扩展算法，而不是仅增加图表或追求更高的单次分数。

14. 参考资料

1. Stanford SNAP. email-Eu-core network 数据集与原始文件。数据来源和部门真值。

2. TuGraph-family. TuGraph v4.5.2 Release。本次实际安装版本。

3. TuGraph. Cypher API。Schema、导入及存储过程接口；具体行为另与 v4.5.2 源码及实际返回核对。

4. TuGraph. Stored Procedure Guide。C++ 存储过程机制。

5. Grover, A., Leskovec, J. node2vec: Scalable Feature Learning for Networks. KDD, 2016。

6. Raghavan, U. N., Albert, R., Kumara, S. Near linear time algorithm to detect community structures in large-scale networks. Physical Review E, 76, 036106, 2007。

7. Page, L., Brin, S., Motwani, R., Winograd, T. The PageRank Citation Ranking: Bringing Order to the Web。PageRank 原理。

8. 课程材料：《专业综合实践-实验2-图查询与图算法》PPT，重点对应第 2、5、9-15、17-27 页；配套 ceshi_node2vec.py。